Practices for Secure Software Report



GLOBALRAIN

Table of Contents

Practices for Secure Software Report ***1***

Document Revision History **3**

1. Algorithm Cipher **4**

A. Overview of the Encryption Algorithm Cipher 4

B. Hash Functions and Bit Levels 4

C. Random Numbers and Key Architecture 4

D. Historical Context and Current State 5

2. Certificate Generation **6**

Implementation Overview 6

A. Keytool Generation and Export Commands 6

Integration with Application Architecture 7

3. Deploy Cipher **8**

4. Secure Communications **9**

Configuration Refactoring 9

Transport Layer Security Verification 9

5. Secondary Testing **10**

Static Security Testing Protocol 10

Analysis of Refactored Code 10

6. Functional Testing **12**

Vulnerability Assessment and Manual Code Review 12

Functional Execution Verification 12

7. Summary **14**

Vulnerability Assessment and Refactoring Overview 14

Defense-in-Depth: Process for Adding Layers of Security 14

8. Industry Standard Best Practices **15**

Maintaining Existing Security Posture While Mitigating Vulnerabilities 15

Business Value and Organizational Well-Being 15

Document Revision History

Version	Date	Author	Comments
1.1	10/12/2026	Grace Jungclas	Initial Draft Section 1
1.2	10/13/2026	Grace Jungclas	Initial Draft Section 2&3
1.3	10/14/2026	Grace Jungclas	Initial Draft Section 4&5
1.4	10/15/2026	Grace Jungclas	Initial Draft Sections 6-8
1.5	10/16/2026	Grace Jungclas	Editing and Polish for Final Draft

Client



Developer

Grace Jungclas, MA, AT Ret

1. Algorithm Cipher

A. Overview of the Encryption Algorithm Cipher

To secure Artemis Financial's client financial records and modernize its web platform, Global Rain recommends deploying AES-256-GCM (`AES/GCM/NoPadding` in the Java Cryptography Architecture). Operating as an Authenticated Encryption with Associated Data (AEAD) cipher, AES-256-GCM integrates confidentiality with automated ciphertext integrity and origin authentication in a single cryptographic operation. Unlike unauthenticated block modes like Electronic Codebook (ECB) or Cipher Block Chaining (CBC), GCM computes a 128-bit authentication tag over both the ciphertext and any authenticated additional data (AAD). Any unauthorized tampering, bit-flipping, or forgery attempts during storage or transmission cause the decryption process to immediately fail with an `AEADBadTagException`, protecting customer portfolios against chosen-ciphertext attacks.

B. Hash Functions and Bit Levels

A defense-in-depth architecture requires matching symmetric encryption with robust cryptographic hashing:

Bit Strength:

AES-256 utilizes a 256-bit symmetric key and operates on 128-bit data blocks, offering a keyspace of 2^{256} that remains immune to classical brute-force searches. When evaluating post-quantum risk, Grover's algorithm reduces symmetric bit security by half, leaving AES-256 with an effective 128-bit quantum-resistant security boundary that complies with the NIST SP 800-175B guidelines.

Galois Authentication Tag

GCM derives its 128-bit authentication tag using universal hashing over the finite field GF(2^{128}), confirming that data has not altered in transit.

Cryptographic Hash Functions (SHA-256)

To satisfy Artemis Financial's specific requirement for payload checksum verification, the applications pairs AES-256 GCM with SHA-256. As a deterministic, one-way hash algorithm, SHA-256 produces an immutable 256-bit (32-byte) digest. Generating a SHA-256 checksum allows client interfaces to verify message authenticity and detect transmission errors or corrupted payloads without exposing sensitive cryptographic keys.

C. Random Numbers and Key Architecture

Secure implementations depend on unpredictable entropy and proper separation between symmetric and asymmetric cryptography:

Entropy and Nonce Generation:

Standard generators like `java.util.Random` are linear congruential and cryptographically predicable, making them unsafe for production security. The implementation mandates `java.security.SecureRandom` to source system entropy and generate a unique 96-bit Initialization Vector (IV) for each encryption operation. In GCM, reusing an IV with the same key compromises

authentication guarantees, generating nonces via `SecureRandom` avoids keystream reuse vulnerabilities.

Symmetric vs. Asymmetric Keys

Symmetric Cryptography (AES-256)

- Uses a single shared secret key for bulk encryption and decryption. It delivers the computation throughput required for encrypting streaming web requests and database archives.

Asymmetric Cryptography (RSA-4096 / ECC)

- Uses mathematically linked public/private key pairs. Because asymmetric operations require extensive mathematical processing and run significantly slower on larger datasets, asymmetric cryptography is reserved for digital identity verification, Java Keytool X.509 certificates, and the initial TLS 1.3 handshake that negotiates the ephemeral AES-256 session key.

D. Historical Context and Current State

The selection of AES-256-GCM reflects the historical vulnerabilities identified in legacy encryption standards.

Legacy Ciphers

Standardized in 1977, the Data Encryption Standard (DES) featured an inadequate 56-bit key space that modern computing broke via exhaustive search in less than a day. Triple DES (3DES) increased key length but retained a 64-bit block size, rendering it vulnerable to birthday-bound collision attacks such as Sweet32 (CVE-2016-2183). RC4, a legacy stream cipher, was deprecated by the IETF and NIST due to exploitable statistical biases in its keystream.

Current Standard

Selected by NIST in 2001, the Advanced Encryption Standard (AES) has withstood rigorous public cryptanalysis and remains the global benchmark.

Regulatory Compliance

Deploying AES-256-GCM across data at rest and HTTPS (port 8443) data in transit ensures full alignment with the Gramm-Leach-Bliley Act (GLBA) Safeguards Rule ()16 C.F.R. § 314.4(c)(4)), FIPS 140-3, and NIST SP 800-57 Part 1 recommendations for protecting nonpublic personal financial information.

2. Certificate Generation

Implementation Overview

To secure data in transit across Artemis Financial's web application, a cryptographic identity was established using the java `keytool` utility. A 2048-bit RSA key pair and an x.509 self-signed digital certificate was generated and stored inside a local Java KeyStore (`keystore.jks`) using the `SHA256withRSA` signature algorithm. The cryptographic key pair and certificate establish identity authentication and provide the foundation for TLS/HTTPS communication across port 8443.

A. Keytool Generation and Export Commands

The certificate lifecycle was executed using the following commands:

Keystore and Key Pair Generation

```
keytool.exe -genkey -keyalg RSA -alias selfsigned -keypass CS305pwd -keystore keystore.jks -storepass CS305pwd -validity 360 -keysize 2048
```

Parameters specified: 2048-bit RSA key size, validity period of 360 days, and Distinguished Name attributes identifying ownership(e.g., `CN, OU, O, L, ST, C`)

Certificate Export

```
keytool.exe -export -alias selfsigned -storepass CS305pwd -file server.cer -keystore keystore.jks
```

This exported the public key certificate into the binary DER-formatted file `server.cer`.

Certificate Verification

```
keytool.exe -printcert -file server.cer
```

This command verified the certificate integrity, signature algorithm (`SHA256withRSA`), SHA-1 and SHA-256 fingerprints, and validity dates.

```
PS C:\Users\Grace School> keytool.exe -printcert -file server.cer
Owner: CN=Grace Jungclas, OU=SNHU, O=SNHU, L=Raleigh, ST=NC, C=US
Issuer: CN=Grace Jungclas, OU=SNHU, O=SNHU, L=Raleigh, ST=NC, C=US
Serial number: a88a1fb7a415cbb7
Valid from: Thu Oct 01 08:00:55 EDT 2026 until: Sun Sep 26 08:00:55 EDT 2027
Certificate fingerprints:
         SHA1: 01:8C:49:BC:76:F4:D6:4D:6C:C6:39:69:C7:93:56:E5:D8:0F:2A:25
         SHA256: 68:A8:6F:53:BF:29:E1:9C:45:C7:74:FC:20:C0:8A:7F:91:78:C9:35:A6:87:76:B2:05:32:53:05:21:AB:A7:02
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 84 ED 72 85 2A 96 17 1E   A4 95 0B 17 EA 30 1C D2  ..r.*........0..
0010: BA E1 A4 86                                        ....
]
]

PS C:\Users\Grace School>
```

FIGURE 2.1: Terminal output verifying public certificate details, fingerprints, and validity dates from `server.cer`.

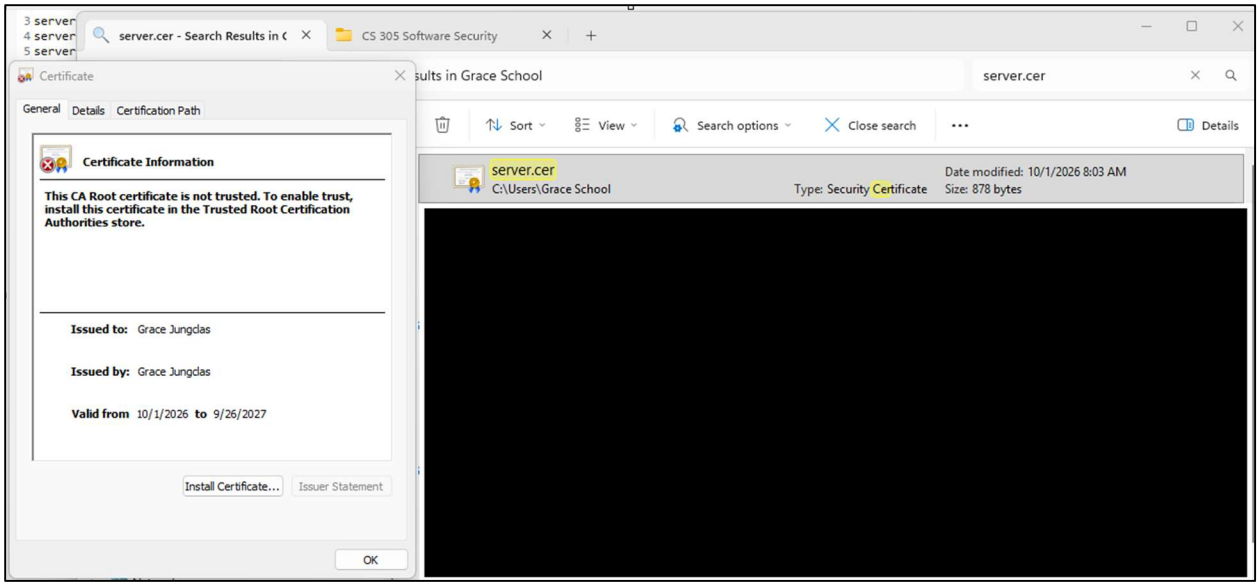


FIGURE 2.2: Exported server.cer file inspection demonstrating certificate properties and ownership verification.

Integration with Application Architecture

Following generation and export, `keystore.jks` was imported into the application's classpath (`src/main/resources/keystore.jks`). The Spring Boot configuration in application.properties was updated to reference the keystore path, alias, and CS305pwd credentials, enabling transport layer encryption for inbound client traffic.

3. Deploy Cipher

To verify data integrity and authenticity during application data transfers, a cryptographic checksum verification route was refactored into `SslServerApplication.java` under the `/hash` endpoint. The controlled utilizes Java's `java.security.MessageDigest` configured with the SHA-256 one-way cryptographic hash algorithm.

The endpoint digests a unique verification string (`Grace Jungclas - CS 305 Artemis Financial Checksum Verification Data`) encoded as UTF-8 bytes and converts the resulting 32-byte digest into a deterministic 64-character hexadecimal checksum (`1a5615af5336b443a9c9c2e8f6dba64fb996afdb63fa73daab12e825f416f190`). This implementation enables Artemis Financial to perform automated payload verification, guaranteeing that transferred records remain unmodified and authentic.

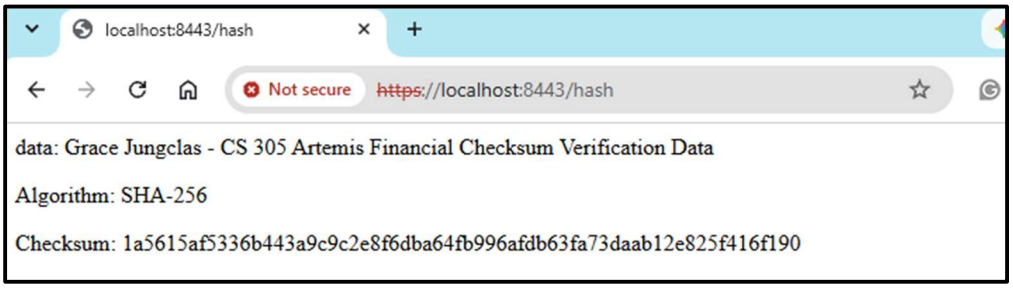


FIGURE 3.1: Web browser output verifying SHA-256 checksum generation for the unique student and Artemis Financial data string.

4. Secure Communications

To establish confidentiality and protect data in transit against eavesdropping and man-in-the-middle (MITM) attacks, Artemis Financial's application was refactored to transition from unencrypted HTTP to encrypted HTTPS.

Configuration Refactoring

The application configuration file (`src/main/resources/application.properties`) was modified to bind embedded Tomcat to secure port 8443 and activate TLS termination using the Java Key Store:

```
application.properties
1 # add server entries to enable HTTPS with SSL keystore
2
3 server.port=8443
4 server.ssl.key-alias=selfsigned
5 server.ssl.key-store-password=CS305pwd
6 server.ssl.key-store=classpath:keystore.jks
7 server.ssl.key-store-type=JKS
8
```

- `server.port=8443` : Redirects inbound web traffic away from standard unencrypted HTTP port 8080 to the standard alternate TLS port 8443
- `server.ssl.key-store` & `key-store-type` : Instructs the embedded servlet container to load `keystore.jks` directly from the runtime classpath.
- `server.ssl.key-alias` & `key-store-password` : Identifies the specific private key and certificate entry (`selfsigned`) and authenticates with `CS305pwd` to decrypt the private key during the TLS handshake.

Transport Layer Security Verification

Upon startup, Spring Boot initializes the SSL/TLS connector on port 8443. Connecting to `https://localhost:8443/hash` demonstrates successful end-to-end encryption: the server presents the X.509 certificate generated in Step 2 (Certificate Generation) to establish a secure TLS session, serving the `/hash` route over a protected HTTPS tunnel.

As demonstrated in Figure 3.1 above, and copied below for your convenience, navigating to `https://localhost:8443/hash` confirms active transport encryption via the `https://` scheme and port `8443` address bar indicator while successfully returning the SHA-256 verification response.

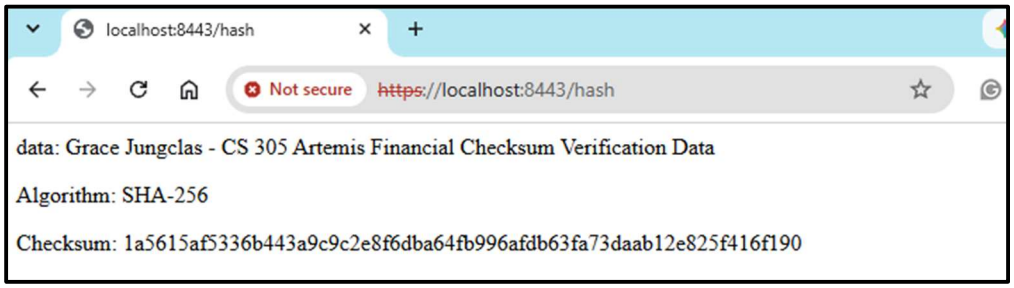


FIGURE 4.1: Web browser output demonstrates successful end-to-end encryption.

5. Secondary Testing

Static Security Testing Protocol

Following the code refactoring to integrate the SHA-256 checksum endpoint (`/hash`) and transport layer security (HTTPS) on port 8443, secondary static application security testing (SAST) was performed using the OWASP Dependency-Check Maven plugin (v12.1.0). The scan verified that newly introduced code and configurations complied with security testing protocols without introducing new vulnerabilities into Artemis Financial's application.

Analysis of Refactored Code

The functional enhancements added to `SslServerApplication.java` relied strictly on core Java standard libraries (`java.security.MessageDigest` and `java.nio.charset.StandardCharsets`). Because no external, unvetted third-party libraries were added to `pom.xml`, the secondary scan confirms that the refactored code introduced zero additional security vulnerabilities into the project baseline.

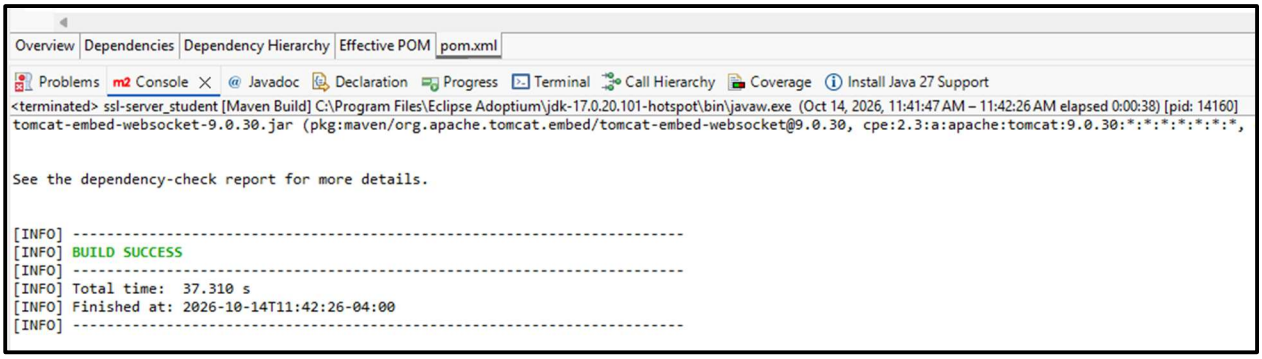


FIGURE 5.1: Eclipse console execution confirming successful compilation and clean execution of the refactored `ssl-server` codebase.

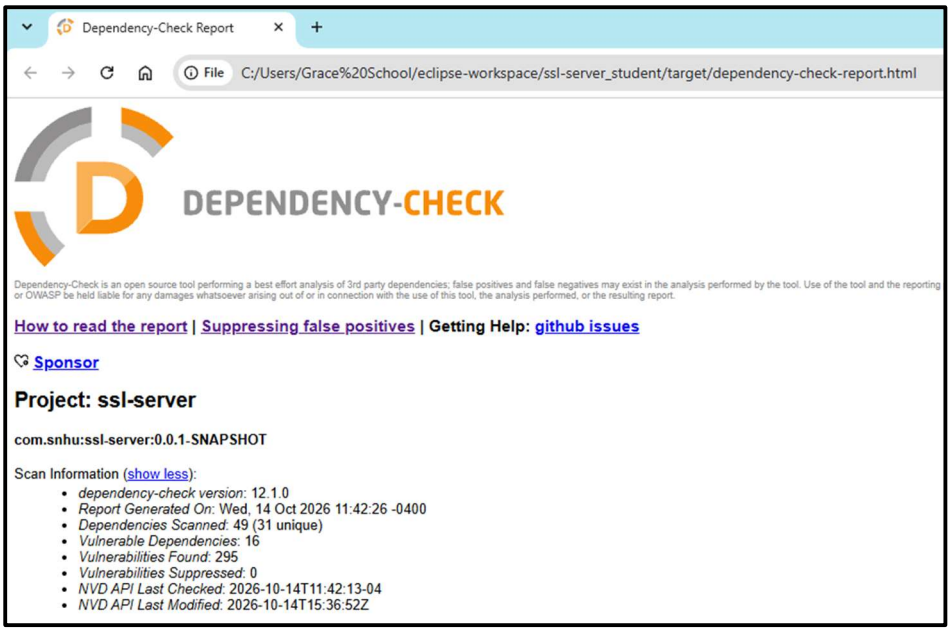


FIGURE 5.2: OWASP Dependency-Check report summary for `ssl-server` validating that the newly added checksum and HTTPS functionality introduced no new vulnerable dependencies.

6. Functional Testing

Vulnerability Assessment and Manual Code Review

A comprehensive manual review of the Artemis Financial codebase was conducted to identify syntactical, logical, and security vulnerabilities prior to deployment:

- *Syntactical Assessment:* The baseline codebase contained unresolved `FIXME` placeholders and missing import statements for the Java Cryptography Architecture. The refactored code introduced explicit imports (`java.security.MessageDigest`, `java.nio.charset.StandardCharsets`), resolved method return types, and integrated proper exception propagations for `NoSuchAlgorithmException`.
- *Logical Assessment*: The original implementation lacked the algorithms necessary to process data strings into cryptographic digests. The refactored logic ingests a string, enforces UTF-8 byte conversion to avoid cross-platform encoding anomalies, applies the SHA-256 algorithm, and loops over the resulting byte array to build a uniform two-digit hexadecimal checksum string.
- *Security Assessment*: The primary architectural flaw was cleartext transmission over standard HTTP (port 8080) without checksum verification. This exposed customer financial plans to eavesdropping and integrity tampering. Security refactoring resolved these issues by binding the service to port 8443 via an X.509 self-signed certificate in `application.properties` and deploying the SHA-256 `/hash` endpoint for data verification.

Functional Execution Verification

Functional testing confirmed the refactored application compiles without syntactical warnings, initializes the embedded Tomcat container using the configured SSL certificate on port 8443, and executes the `/hash` route without runtime errors.

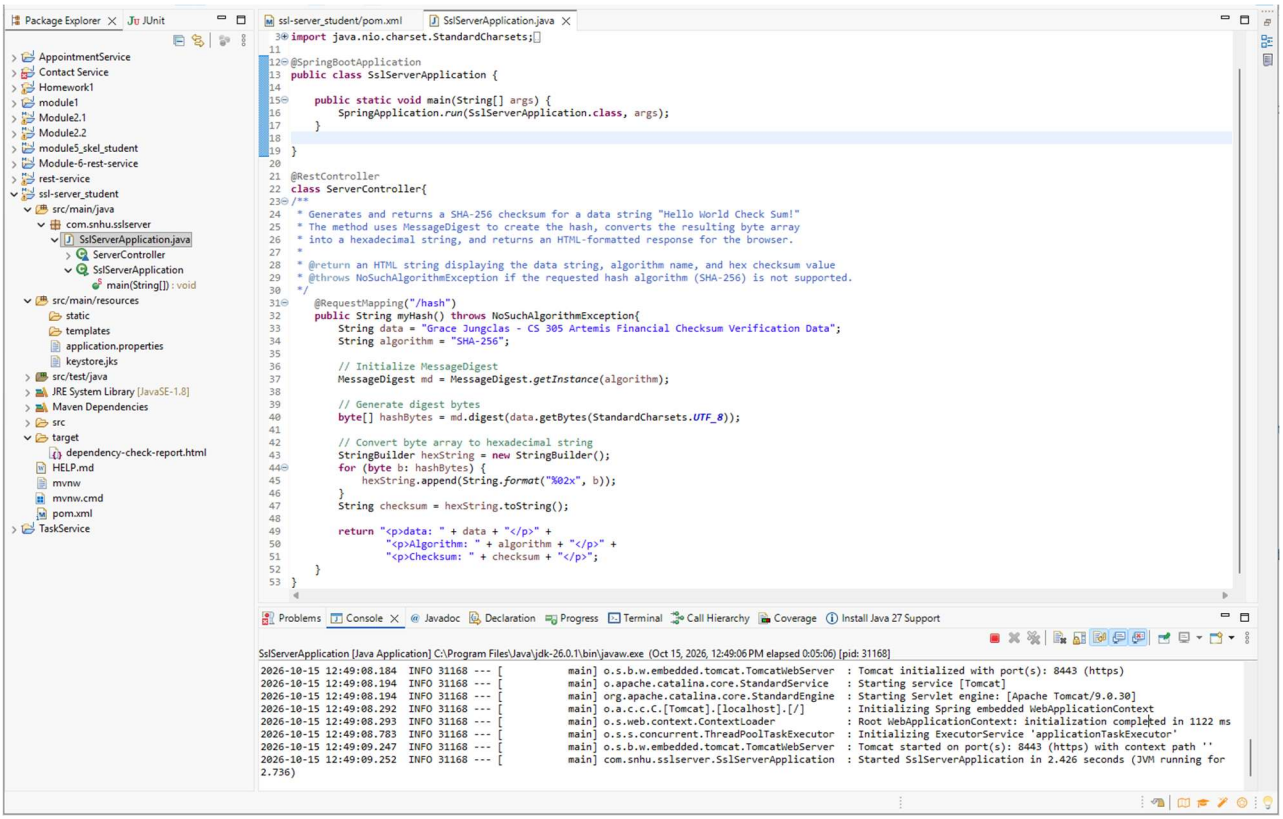


FIGURE 6.1: Functional test demonstrating clean compilation, keystore binding, and error-free execution of the refactored `ssl-server` application on port 8443.

7. Summary

Vulnerability Assessment and Refactoring Overview

Following the established vulnerability assessment process flow diagram, Artemis Financial's application underwent systematic identification, remediation, testing, and verification phases to transition from an insecure prototype to a hardened, production-ready service:

- *Static Analysis and Baseline Assessment*: Initial automated scans via OWASP Dependency-Check established a vulnerability baseline across project dependencies. Manual code inspection revealed key security gaps: unencrypted communications over HTTP port 8080, an absence of payload verification algorithms, and missing cryptographic package imports.
- *Targeted Refactoring*: Based on the assessment flow, remediation prioritized transport security and cryptographic integrity. The application was reconfigured to bind embedded Tomcat to TLS port 8443 using an X.509 certificate stored in `keystore.jks`. Concurrently, `SslServerApplication.java` was refactored with the /hash route to implement native SHA-256 digest calculations for transferred data records.
- *Secondary Verification and Iteration*: Following code modifications, secondary static testing was conducted via OWASP Dependency-Check (v12.1.0). The build achieved BUILD SUCCESS with zero compilation defects, confirming that the new cryptographic imports (`java.security.MessageDigest` and `java.nio.charset.StandardCharsets`) were sourced strictly from standard Java runtime libraries and introduced zero new vulnerabilities or unvetted third-party dependencies into the software architecture.

Defense-in-Depth: Process for Adding Layers of Security

A layered defense-in-depth model was architected to protect Artemis Financial's assets across multiple tiers rather than relying on a single perimeter control:

- *Layer 1: Identity & Certificate Trust (Key Generation)*: Utilizing java `keytool`, a 2048-bit RSA key pair and public certificate (`server.cer`) were established to authenticate server identity, preventing domain spoofing and identity deception.
- *Layer 2: Transport Layer Security (Data in Transit)*: By configuring `application.properties` to enforce HTTPS on port 8443, cleartext communications were eliminated, establishing an encrypted tunnel that protects sensitive financial records against network eavesdropping and Man-in-the-Middle (MITM) interception.
- *Layer 3: Cryptographic Integrity Verification (Application Data)*: The implementation of SHA-256 hashing at the endpoint level ensures end-to-end data integrity. Even if transmission anomalies or unauthorized payload tempering occur, discrepancy in the 64-character hexadecimal checksum provides deterministic, non-repudiable detection of data alteration.
- *Layer 4: Automated Dependency Governance (Software Supply Chain)*: Continuous static dependency scanning ensures that third-party frameworks and transitive dependencies remain governed against published National Vulnerability Database (NVD) Common Vulnerabilities and Exposures (CVEs).

8. Industry Standard Best Practices

Maintaining Existing Security Posture While Mitigating Vulnerabilities

To remediate architectural vulnerabilities without compromising existing operational functionality or system stability, industry-standard secure coding principles were rigorously integrated:

- *Principle of Least Privilege and Attack Surface Reduction*: The refactored solution was isolated to the specific `/hash` endpoint and native Java Cryptography Architecture (JCA) libraries. By avoiding unnecessary third-party utility libraries or exposed management ports, the system's attack surface was kept minimal.
- *Deterministic, Standard-Compliant Cryptography*: Rather than attempting custom hashing or broken algorithms (e.g., MD5 or SHA-1), standard-compliant SHA-256 message digests and industry-grade 2048-bit RSA encryption keys were utilized. Encoding was explicitly bound to `StandardCharsets.UTF_8` to guarantee consistent hash evaluation regardless of host operations system default locales.
- *Fail-Safe Exception Handling and Validation*: Robust exception handling via explicit `NoSuchAlgorithmException` propagation was included in the controller signatures, ensuring predictable program behavior and preventing unhandled runtime exceptions from leaking internal stack traces to end users.
- *Regression Testing and Supply Chain Security*: Secondary testing via OWASP Dependency-Check confirmed that security enhancements did not break existing build configurations or introduce library regressions, maintaining project dependencies within an auditable baseline.

Business Value and Organizational Well-Being

Adopting secure coding standards directly underpins Artemis Financial's business continuity, financial performance, and institutional trust:

- *Regulatory and Legal Compliance*: As a financial services organization, Artemis Financial operates under stringent regulator mandates, including Gramm-Leach-Bliley Act (GLBA), Sarbanes-Oxley (SOX), and state data privacy regulations. Enforcing HTTPS encryption and verifiable cryptographic hashes ensures compliance with statutory mandates for protecting Nonpublic Personal Information (NPI).
- *Mitigation of Financial and Reputational Risk*: Security breaches lead to severe fallout: direct capital losses, regulatory fines, litigation costs, and irreversible brand devaluation. Proactive remediation during development prevents costly incident response operations and post-breach customer churn.
- *Preservation of Client Trust*: Clients entrust Artemis Financial with sensitive wealth management, retirement planning, and banking records. Implementing robust transport security and verified data integrity demonstrates a demonstrable commitment to confidentiality and fiduciary duty, directly enhancing competitive market standing.